

Week 7 In-Class Exercises

(Computational Efficiency and Review)

Part I: Computational Efficiency

Q1: Count Operations [*/**]

Given the following code snippets, assume that **my_list_1** has **m** elements and **my_list_2** has **n** elements, and both lists contain numbers.

(a) How many addition operations (i.e., Line 4 and Line 6) are executed in terms of **m** and **n**?

```
def add_numbers_1(my_list_1, my_list_2):
    total = 0
    for element in my_list_1:
        total = total + element
    for element in my_list_2:
        total = total + element
    return total
```

(b) How many addition operations (i.e., Line 5 and Line 6) are executed in terms of **m** and **n**?

```
def add_numbers_2(my_list_1, my_list_2):
    total = 0
    for element1 in my_list_1:
        for element2 in my_list_2:
            total = total + element1
            total = total + element2
    return total
```

(c) Find the worst-case complexity (Big-O) for the below code, in which **my_list_2** can be assumed to contain at least 100 elements.

```
def add_numbers_3(my_list_1, my_list_2):
    total = 0
    my_list_3 = my_list_2[:100]
    for element1 in my_list_1:
        total = total + element1
        for element2 in my_list_3:
            total = total + element2
    return total
```

Q2: Linear Search [**/**]

You are given a list of student records, where each record is a list containing information about a student, in particular their name (max 10 characters), age (a positive integer < 100), and grade (a single character). Implement a function that performs a search of linear complexity to find and return a list of students who meet all the specified criteria. The function takes the following parameters:

- `students`: list of student records, where each record is a list of [name, age, grade] in this order.
- `criteria`: list of criteria values to match.
- `criteria_indices`: list of indices corresponding to the criteria in the student records.

Example:

```
students = [
    ["Alice", 20, "A"],
    ["Bob", 22, "B"],
    ["Charlie", 20, "A"],
    ["David", 23, "C"]
]
criteria = ["A", 20]
criteria_indices = [2, 1] # Indices corresponding to grade and age
find_students(students, criteria, criteria_indices) returns [['Alice', 20, 'A'], ['Charlie', 20, 'A']]
```

Another example:

```
criteria = [20]
criteria_indices = [1] # Index corresponding to age
find_students(students, criteria, criteria_indices) returns [['Alice', 20, 'A'], ['Charlie', 20, 'A']]
```

Assume all inputs are valid and are not case sensitive. The indices in the `criteria_indices` can be in any order.

Part II: Review

Q1: Code Tracing [*/**]

Given the following pieces of code, what are the outputs? Use memory state diagram to help you.

(a)

```
def do_trick(a_list):  
    a_list = a_list + a_list[1:3] + [a_list[3]]  
    print(a_list)  
  
my_list = ['a', 'b', 'c', 'd']  
do_trick(my_list)  
print(my_list)
```

(b)

```
def do_trick(a_list):  
    a_list.append(a_list[1:3] + [a_list[3]])  
    print(a_list)  
  
my_list = ['a', 'b', 'c', 'd']  
do_trick(my_list)  
print(my_list)
```

(c)

```
my_str = '123'  
  
for index in range(0, len(my_str)-1):  
    n = int(my_str[index] + my_str[index+1])  
    print(n)  
    m = int(my_str[index:index+2])  
    print(m)
```

Q2: String Transformation [*/**]

Write a function called `transform_string()`. The function takes in a string as its parameter. The function transforms the string as follows:

- Any uppercase letter is replaced by the character ' `␣` ' to indicate that it's a letter in uppercase.
- Any lowercase letter is replaced by the character ' `␣` ' to indicate that it's a letter in lowercase.
- Any digit is replaced by the character ' `␣` ' to indicate that it's a digit.
- Any other character (e.g., a space, a semi-colon, etc.) is replaced by the character ' `␣` ' to indicate that it's symbol.

The function **returns** the transformed string. The function **also prints out** the numbers of each group of characters. For example, when we run `print(transform_string("IS1704 is a fun module"))`, we should see the following output:

```
Number of uppercase letters: 2
Number of lowercase letters: 12
Number of digits: 4
Number of symbols: 4
LLddddsllslslllsllllll
```

Q3: Triangle and Frame [**]

- (a) Write a function called `print_triangle(ch, num_rows)` that takes in a character and a number as its parameters. The number indicates the number of rows. The function ***prints out*** a triangle using the character.

For example, if `ch` is `'*'`, `num_rows` is 3, the following output is to be generated:

```
*
**
***
```

If `ch` is `'#'`, `num_rows` is 5, the following output is to be generated:

```
#
###
####
#####
#####
```

- (b) Write another function called `print_frame(ch, num_rows, num_cols)` that takes the same character and row count as above, along with the number of columns as the third parameter. Assume `num_rows` and `num_cols` are **greater than 1**. The function ***prints out*** a frame as shown below. For example, if `ch` is `'*'`, `num_rows` is 3 and `num_cols` is 4, the following output is to be generated:

```
****
*  *
****
```

If `ch` is `'#'`, `num_rows` is 5 and `num_cols` is 6, the following output is to be generated:

```
#####
#      #
#      #
#      #
#####
```

If `ch` is `'#'`, `num_rows` is 2 and `num_cols` is 2, the following output is to be generated:

```
##
##
```

Q4: Pangram [**]

Implement a function called `is_pangram` to determine if a sentence is a pangram. A pangram is a sentence using every letter of the English alphabet at least once (either in uppercase or in lowercase). The best-known pangram is:

"The quick brown fox jumps over the lazy dog."

The function returns `True` if the given sentence is a pangram or `False` if it is not. For example:

- `is_pangram("The quick brown fox jumps over the lazy cat.")` returns `False`
- `is_pangram("Pack my box with five dozen liquor jugs.")` returns `True`
- `is_pangram("The five boxing wizards jump quickly.")` returns `True`
- `is_pangram("Mr. Jock, TV quiz Ph.D., bags few lynx.")` returns `True`